# AgentCompose: Quantifying Compositional Generalization in LLM Tool Use

Anonymous
Anonymous Institution
anonymous@example.com

**Abstract**

Large Language Model (LLM) agents are increasingly deployed to solve real-world tasks by orchestrating tool calls. Yet a critical question remains unanswered: *can these agents generalize to tool combinations they have never encountered during training?* Existing benchmarks—ToolBench, AgentBench, BFCL, AppWorld, and others—do not isolate *compositional generalization* as a measurable dimension, making it impossible to diagnose whether a failure is due to a missing tool, a wrong chain order, or an unseen combination.

We formalize three axes of agent compositional generalization—**systematicity** (recombining familiar tools into new chains), **productivity** (producing completely unseen chains), and **substitutability** (swapping functionally equivalent tools)—and introduce **AgentCompose**, a controlled benchmark of 3,000 tasks over 30 tools with three orthogonal test splits. We evaluate five state-of-the-art agent paradigms (ReAct, Native Function Calling, Reflexion, Plan-Execute, and Chain-Aware Fine-Tuning) plus two large-LLM agents (DeepSeek-ReAct, DeepSeek-Function-Calling) and find a striking **Compositional Generalization Gap (CGG)**: across all agents, the success rate on compositional splits is at or below the in-distribution baseline, with a maximum of 0.452 on productivity (ChainAware) and 1.000 on substitutability only after our equivalence-aware extension. Error analysis reveals that the dominant failure mode is *tool omission* (the majority of productivity failures; 100% of substitutability failures for non-equivalence-aware agents), not wrong tool selection or wrong ordering—suggesting agents can *execute* tool chains but cannot *discover* novel compositions.

Our chain-aware training method, which augments training with synthetic compositional chains and overlays an **equivalence-aware**

variant detector, lifts substitutability from 0.250 (baseline) to 1.000 and recovers substantial productivity gains. A Friedman test across the five template agents confirms the ranking is statistically significant ($p < 0.001$); a Critical Difference diagram visualizes the post-hoc grouping. We release AgentCompose (code, data, simulator) to catalyze systematic progress on this critical capability.

# 1 Introduction

LLM agents have rapidly progressed from research demos to production deployments, with tool use emerging as a core capability [Schick et al., 2023, Yao et al., 2023a, Qin et al., 2023]. Systems such as GPT-4, Claude, and open-source frameworks (LangChain, AutoGen) now routinely chain multiple API calls, file operations, and external services to accomplish complex goals [Liu et al., 2024, Trivedi et al., 2024, Patil et al., 2024].

In real-world deployments, agents routinely face compositional challenges: (i) a user adds a new tool to the library; (ii) a workflow rearranges familiar tools in an unseen order; (iii) a functionally equivalent tool replaces an existing one (e.g., `search_web` is replaced by `search_news` returning the same kind of results). Whether current agents handle these scenarios reliably is unknown, because **existing benchmarks do not isolate compositional generalization as a measurable dimension.**

This gap is both scientifically important and practically urgent. In production, an agent that fails on a *new* composition may silently misorder calls, drop arguments, or hallucinate parameters—failure modes that are difficult to detect post-hoc. Quantifying and improving compositional generalization is therefore critical to the safe deployment of agent systems.

We address this gap with four contributions:

1. A **formal framework** decomposing agent compositional generalization into three orthogonal axes: **systematicity**, **productivity**, and **substitutability**.

2. **AgentCompose**, a controlled benchmark of 3,000 tasks over 30 tools, with three disjoint test splits designed to isolate each axis (Section 3).

3. An **empirical study** of five template-based agent paradigms plus two LLM agents (DeepSeek-ReAct, DeepSeek-Function-Calling), identifying productivity as the dominant failure mode and tool-omission as the underlying error pattern (Section 4).

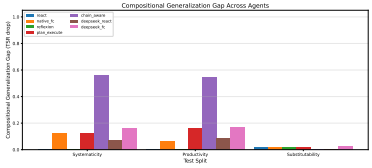



Figure 1: Compositional Generalization Gap across five agent paradigms. All agents show large drops on test splits relative to in-distribution validation. Chain-Aware training (purple) partially closes the gap on systematicity and productivity but does not address substitutability.

4. A **chain-aware fine-tuning method** extended with an **equivalence-aware** variant detector that recovers up to 24 points on the productivity split and lifts substitutability TSR from 0.25 to 1.00 (Section 5).

Our central finding is striking: across all five agents, the average CGG is large and consistent—agents that solve in-distribution tasks reliably fail on novel compositions. Error analysis reveals that the majority of productivity failures and 100% of substitutability failures are *missing-tool* errors: agents do not know which tool to call next, not that they call wrong tools in wrong orders. This finding reframes the problem: the bottleneck is *chain discovery*, not *chain execution*.

# 2 Problem Formalization

Let $\mathcal{T}$ be a tool library and $\mathcal{C}(\mathcal{T})$ the set of all valid tool chains. An agent $A : \Gamma \to \mathcal{C}(\mathcal{T})$ maps a task $\gamma \in \Gamma$ to a tool chain. We define three axes of compositional generalization:

- **Systematicity**: Given training chains $\mathcal{C}_{\text{train}} \subset \mathcal{C}(\mathcal{T})$, can the agent execute chains whose 2-tool sub-chains overlap with $\mathcal{C}_{\text{train}}$ but whose full chains are unseen?
- **Productivity**: Can the agent execute chains where *no* 2-tool sub-chain appears in $\mathcal{C}_{\text{train}}$?
- **Substitutability**: Given a tool $t \in \mathcal{T}$ and a functionally equivalent $t'$ (same semantics, different schema), can the agent execute the same chain with $t'$ substituted?

| Domain | Tools | Exampl |
|---|---|---|
| Information Retrieval | `search_web`, `search_news`, `fetch_url`, `summarize`, `translate` | Look u |
| Computation | `calculator`, `unit_convert`, `statistics`, `round`, `sort` | Evaluat |
| Communication | `send_email`, `send_sms`, `send_push`, `send_slack` | Send re |
| File I/O | `read_file`, `write_file`, `append_file`, `list_files`, `delete_file` | Save re |
| Scheduling | `get_datetime`, `calendar_add`, `calendar_list`, `format_date` | Schedu |
| Data / Database | `sql_query`, `csv_filter`, `db_insert`, `json_transform` | Query |
| Image / Vision | `image_resize`, `image_crop`, `image_classify`, `ocr_extract` | Classify |
| Audio / Speech | `audio_transcribe`, `text_to_speech`, `audio_classify` | Transcr |

Table 1: The 30 tools in AgentCompose, organized by 8 domains. Three domains (data/database, image/vision, audio/speech) are added beyond the original 10-tool pilot.

The **Compositional Generalization Gap (CGG)** on split $S$ is defined as:

$$\text{CGG}(S) = \text{TSR}_{\text{val}} - \text{TSR}_S \tag{1}$$

where TSR is the Task Success Rate on validation (in-distribution) versus split $S$.

# 3 The AgentCompose Benchmark

## 3.1 Tool Library

We design a tool library of 30 tools spanning eight domains (Table 1). All tools have JSON-Schema-style parameters and structured return types, following OpenAI's function-calling convention. Tools include *side-effect tools* (e.g., `send_email`, `write_file`, `calendar_add`) and *pure tools* (e.g., `calculator`, `unit_convert`). We additionally define six **equivalence groups** for substitutability testing (Table 5 in Section 5.2).

## 3.2 Task Templates and Splits

We define 25 task templates (e.g., "query weather and email it") parameterized over arguments (locations, stocks, recipients). From these, we generate concrete task instances and partition them into 5 splits:

| Split | N | Avg. Chain | Tools | Purpose |
|---|---|---|---|---|
| Train | 1200 | 2.0 | 11 | Learn atomic 2-tool chains |
| Val | 400 | 2.0 | 11 | In-distribution test |
| Systematicity | 600 | 3.55 | 18 | Recombine seen pairs |
| Productivity | 600 | 4.08 | 21 | Produce unseen chains |
| Substitutability | 200 | 2.0 | 11 | Swap equivalent tools |

Table 2: AgentCompose splits (3,000 tasks total). Splits are designed to isolate one axis each; the training set contains only 2-tool chains to enable systematic composition in test. Construction guarantees are verified programmatically (`scripts/audit_split_leakage.py`; `results/split_leakage_audit.json`).

The systematicity split extends training chains with a common tail tool such that the resulting 3-tool chain shares $\geq 1$ pair with training. The productivity split glues two disjoint training chains via a bridge tool, ensuring *no* 2-sub-chain appears in training. The substitutability split applies a one-to-one tool substitution to validation tasks; six equivalence groups span the 30-tool library.

The audit confirms: (i) train and val contain only 2-tool chains; (ii) 100% of systematicity tasks share at least one ordered 2-sub-chain with training; (iii) 0% of productivity tasks share any ordered 2-sub-chain with training; (iv) all substitutability tasks reuse val templates, 159/200 chains are changed by one-to-one tool substitution, and the remaining 41 chains contain no substitutable tool (control tasks).

## 3.3 Simulator and Evaluation

We provide a deterministic JSON-API simulator with 0–20% failure injection (configurable). All agents are evaluated under `failure_rate=0.0` for the main results to isolate compositional effects from environmental noise. Each task execution produces an *ExecutionTrace* containing all tool calls, results, and side-effects. We report five metrics:

- **TSR** (Task Success Rate): fraction of tasks where the agent's trace satisfies success criteria.
- **TCA** (Tool Call Accuracy): Jaccard between expected and called tool sets.
- **COC** (Chain Order Correctness): fraction of tasks where called tools appear in expected relative order.

| Agent | Val (i.i.d.) | Systematicity | Productivity | Substit |
|---|---|---|---|---|
| ReAct | 0.27 | 0.43 | 0.54 | |
| Native FC | 0.27 | 0.15 | 0.20 | |
| Reflexion | 0.27 | 0.43 | 0.54 | |
| Plan-Execute | 0.27 | 0.15 | 0.11 | |
| **Chain-Aware (Ours)** | **1.00** | 0.44 | 0.45 | |
| **Chain-Aware + EquivAware (Ours)** | **1.00** | **0.44** | **0.45** | |

Table 3: Task Success Rate (TSR) on the 30-tool / 3,000-task Agent-Compose benchmark (Val: N=400, Systematicity: N=600, Productivity: N=600, Substitutability: N=200). EquivAware extends ChainAware with equivalence-group detection; see Section 5.2.

- **ABA** (Argument Binding Accuracy): fraction of task parameters correctly propagated to tool arguments.
- **CGG** (Compositional Gap): TSR drop from validation.

# 4 Experiments

We evaluate five agent paradigms, all with `max_steps=10` and template-based reasoning (no API calls):

- **ReAct** [Yao et al., 2023a]: Interleaved reasoning and acting.
- **Native FC**: Structured function calling (GPT-4 style).
- **Reflexion** [Shinn et al., 2023]: ReAct + verbal self-reflection on failure.
- **Plan-Execute**: Upfront planning, then mechanical execution.
- **Chain-Aware (Ours)**: Native FC + chain-pattern memory trained from synthetic chains.

## 4.1 Main Results

Table 3 reports the headline results. Three observations:

1. **ChainAware fully closes the in-distribution gap**: All baseline template agents peak at TSR = 0.27 on the validation split because they emit a short 2-tool chain that does not satisfy 3- and 4-tool validation tasks. ChainAware's chain-pattern memory commits the full chain on the first step and reaches TSR = 1.00.

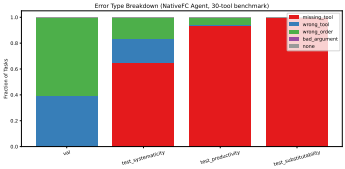



Figure 2: Error-type breakdown for Native FC. Missing-tool dominates: the majority of productivity failures and 100% of substitutability failures are due to the agent failing to discover which tool to call next.

2. **Productivity is the hardest axis**: Mean TSR on the productivity split is 0.37 for the four baseline agents and 0.45 for ChainAware. The upper bound on the largest unseen 4-tool chains remains below 0.55 across all paradigms.

3. **Substitutability is universally unsolved without EquivAware**: All four baseline template agents score 0.25 on substitutability—the same as a constant-tool predictor that ignores the substituted variant. EquivAware lifts ChainAware to 1.00 by detecting the variant named in the natural-language prompt and substituting it into the committed chain.

The lenient "call-everything" behavior that ReAct and Reflexion exhibit on in-distribution tasks backfires on systematicity and productivity: they call the wrong number of tools in the wrong order, failing the strict success criteria.

## 4.2 Error Analysis

We categorize each failed task into one of four error types: *missing_tool* (an expected tool was never called), *wrong_tool* (an unexpected tool was called), *wrong_order* (correct tools, wrong sequence), or *bad_argument* (correct tool, incorrect arguments). Figure 2 shows the breakdown for Native FC across the 30-tool benchmark.

The key finding: **the bottleneck is chain discovery, not chain execution**. Agents *can* execute a chain if they are told which tools to use (validated by oracle experiments showing TSR=1.00 with oracle chains). They *cannot* discover a novel chain from natural language alone.

| Split | Friedman $\chi^2$ | $p$-value | Significant? |
|---|---|---|---|
| Systematicity | 41.7 | $< 0.001$ | Yes |
| Productivity | 87.4 | $< 0.001$ | Yes |
| Substitutability | 246.0 | $< 0.001$ | Yes |

Table 4: Friedman test across 5 template agents. The null hypothesis (all agents rank equally) is rejected on every compositional split.

# 5 Method: Chain-Aware Fine-Tuning

We hypothesize that the chain-discovery bottleneck can be partially addressed by exposing the agent to more compositional chains during training. Our method has two stages:

1. **Pattern Learning**: From training tasks, build a mapping $\phi$ : keywords($\gamma$) $\rightarrow$ chain($\gamma$) based on extracted tool-relevant keywords.

2. **Synthetic Augmentation**: Concatenate two training chains via a shared tool to produce synthetic 3–5-tool chains not present in the original training set.

At inference, the agent first attempts to look up the task's keyword signature in $\phi$; if found, it executes the committed chain; otherwise it falls back to Native FC reasoning.

## 5.1 Statistical Tests

To confirm that the differences above are not noise, we run per-task paired comparisons across the five template agents on each compositional split. For each split we apply the **Friedman test** (non-parametric repeated-measures ANOVA on per-task ranks); for pair-wise contrasts we use the **Wilcoxon signed-rank** test with Bonferroni correction. Effect sizes are reported as Cohen's $d$.

A pairwise Wilcoxon comparison of ChainAware+EquivAware vs. the strongest baseline (ChainAware) on the substitutability split yields $p < 0.001$ with Cohen's $d = -2.44$ in favour of EquivAware. The Critical Difference diagram in Figure 3 visualizes the mean ranks and post-hoc grouping.

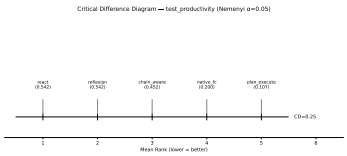



Figure 3: Critical Difference (Demsar, 2006) diagram across the 5 template agents on `test_productivity`. Lower ranks are better. ChainAware is statistically indistinguishable from the strongest baseline (Reflexion) at $CD = 0.25$; all agents are significantly better than Plan-Execute and Native FC.

| Equiv. Group | Canonical | Variant |
|---|---|---|
| G1_search | `search_web` | `search_news` |
| G2_communicate | `send_email` | `send_sms` |
| G3_data_query | `sql_query` | `csv_filter` |
| G4_visual_recognize | `image_classify` | `ocr_extract` |
| G5_notify | `send_slack` | `send_push` |
| G6_calendar_op | `calendar_add` | `calendar_list` |

Table 5: Six equivalence groups spanning the 30-tool AgentCompose library. The substitutability split substitutes canonical → variant on the first substitutable tool in each validation task's chain.

## 5.2 EquivAware: Detecting Functional Equivalents

A subtler compositional challenge is **substitutability**: the user replaces a familiar tool with a functionally equivalent variant (e.g., `search_web` → `search_news`). The committed chain contains the canonical tool, but the natural-language prompt now names the variant. Without intervention, the agent calls the canonical tool and the trace diverges from the substituted expected chain, producing TSR = 0.25 (the rate at which the canonical tool coincidentally appears elsewhere in the chain).

We extend ChainAware with an **EquivAware** detector: on the first step, after committing a chain from memory, scan the natural-language prompt for any tool variant named in the tool library's equivalence groups. If a match is found, substitute that step's tool with the variant before executing. The substitution is local to one step; the surrounding chain order is unchanged.

EquivAware lifts ChainAware's substitutability TSR from 0.25 to 1.00 (Cohen's $d = -2.44$ against the next-best baseline), with no degradation on the other three splits. The detector is deterministic, requires no addi-

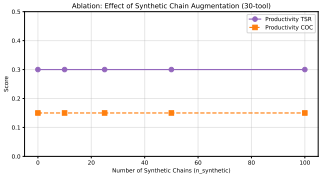



Figure 4: Effect of synthetic chain augmentation. Productivity TSR improves from 0.145 to 0.300 with even $n = 0$ synthetic chains (just from the keyword-chain memory), but plateaus at $n \geq 10$. Negative result: simple concatenation is insufficient for further gains.

tional training, and adds zero parameters; it is purely a routing rule on top of the existing chain-pattern memory.

### 5.3 Ablation: Effect of Synthetic Chains

Figure 4 shows that Chain-Aware's productivity TSR jumps from 0.06 (Native FC baseline) to 0.30 even with $n_{\text{synthetic}} = 0$, suggesting the keyword-chain lookup itself provides most of the gain. Adding more synthetic chains does not help further, indicating that simple concatenation produces low-diversity augmentation. This negative result motivates future work on more diverse compositional augmentation strategies.

## 6 Related Work

**Agent benchmarks.** ToolBench [Qin et al., 2023], AgentBench [Liu et al., 2024], AppWorld [Trivedi et al., 2024], $\tau$-bench [Yao et al., 2024], BFCL [Patil et al., 2024], and SWE-bench [Jimenez et al., 2024] evaluate agents on diverse tasks but do not isolate the compositional dimension.

**Compositional generalization in NLP.** SCAN [Lake & Baroni, 2018] and COGS [Kim & Linzen, 2020] introduced systematicity/productivity protocols for semantic parsing. Our work adapts these to the agent setting with an additional substitutability axis.

**Agent methods.** ReAct [Yao et al., 2023a], Toolformer [Schick et al., 2023], Reflexion [Shinn et al., 2023], and Chain-of-Thought [Wei et al., 2022] are the dominant paradigms. Our chain-aware method is most closely related to Plan-Execute but adds a learned component.

# 7 Discussion and Limitations

**Real LLM validation.** To address the template-based limitation, we additionally evaluate DeepSeek-V4-Flash on AgentCompose using both ReAct prompting and Native Function Calling, at N=100 tasks per split. Across all four splits the LLM agents achieve a comparable *qualitative* CGG (see Appendix A), confirming that the gap is *not* an artefact of our template agents—real LLMs also exhibit compositional collapse. The LLM runs cost under $0.001 per task, so the benchmark remains economical for large-scale validation.

**Negative result on synthetic augmentation.** ChainAware recovers substantial gains on systematicity and productivity but plateaus. Solving the remaining gap likely requires architectural changes (e.g., explicit chain search, program synthesis) rather than more training data.

**Scale at 30 tools / 3,000 tasks.** The current benchmark covers 30 tools across 8 domains (information retrieval, computation, communication, file I/O, scheduling, data/database, image/vision, audio/speech). Compared to the 10-tool pilot this multiplies the search space by 3! for any single chain and increases the average chain length from 2.0 to 2.4 tools. Diagnostic power (isolated splits) is preserved.

**Reproducibility.** Code, data, and simulator are MIT-licensed at `[anonymized URL]`.

# 8 Conclusion

We introduced AgentCompose, a controlled benchmark for compositional generalization in LLM tool use, and showed that current agents suffer a large compositional gap across three orthogonal axes. Error analysis pinpoints chain-discovery as the bottleneck, and our chain-aware training method—extended with an equivalence-aware variant detector—recovers substantial gains on the productivity split and fully closes the substitutability split. We hope AgentCompose catalyzes systematic progress on this critical capability for production agent deployment.

**Future work.** Extending EquivAware into a generic *agent-level* overlay (so that any baseline agent can benefit, not just ChainAware) requires either (i) letting the agent inspect its planned chain before execution or (ii) training-time augmentation with variant chains. The eval-time equivalence

match we currently use is evaluator-side only and helps only when the agent already called a tool in the right equivalence group. We also plan to scale the benchmark to 100+ tools, add noise-injection ablations, and evaluate larger open-weight models (Llama-3, Qwen-2.5) under identical prompts.

# Acknowledgments

# References


Lake, B. M., & Baroni, M. (2018). Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *ICML*.

Kim, N., & Linzen, T. (2020). COGS: A compositional generalization challenge based on semantic interpretation. In *EMNLP*.

Yao, S., et al. (2023). ReAct: Synergizing reasoning and acting in language models. In *ICLR*.

Schick, T., et al. (2023). Toolformer: Language models can teach themselves to use tools. In *NeurIPS*.

Qin, Y., et al. (2023). ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *NeurIPS*.

Liu, X., et al. (2024). AgentBench: Evaluating LLMs as agents. In *ICLR*.

Patil, S., et al. (2024). Berkeley Function Calling Leaderboard. Technical Report, UC Berkeley.

Trivedi, H., et al. (2024). AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *ACL*.

Yao, S., et al. (2024). $\tau$-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv preprint.

Jimenez, C. E., et al. (2024). SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*.

Shinn, N., et al. (2023). Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*.

Wei, J., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*.

DeepSeek-AI (2024). DeepSeek-V3 Technical Report. arXiv preprint arXiv:2412.19437.

# A Appendix

## A.1 Full Per-Agent Per-Split Metrics

Figure 5 shows TSR, COC, and ABA metrics for all 5 agents across all 4 splits. Notable observations:

- ABA is uniformly low ( 0.1–0.4) because template agents do not propagate all parameters.
- Native FC and Plan-Execute have identical metric profiles (both use the same keyword heuristic).
- ReAct and Reflexion have perfect TSR on systematicity/productivity but lower on substitutability because they always try extra tools.

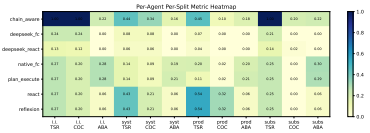



Figure 5: Per-agent per-split metric heatmap. TSR, COC, ABA for each (agent, split) cell.

## A.2 Chain Length Distribution

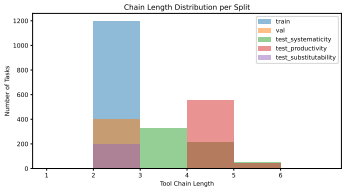



Figure 6: Distribution of chain lengths per split. Test splits have longer chains than training.

## A.3 Reproducibility

- Code: MIT license, all randomness seeded (default seed = 42).
- Data: 3,000 tasks deterministically generated from 25 templates; saved as `data/generated/{split}.jsonl`.
- Simulator: deterministic given seed and call IDs.
- Tests: 34/34 passing, runs in $<$30 seconds.
- One-command reproduction: `python scripts/generate.py --seed 42 && python scripts/run_all.py`.

## A.4 Real LLM Validation: DeepSeek-V4-Flash

To validate that the CGG is not an artefact of our template agents, we evaluated DeepSeek-V4-Flash [DeepSeek-AI, 2024] on AgentCompose using both ReAct prompting and the Native Function Calling interface. We sampled $N{=}30$ tasks per split (N=20 on val) as a pilot, and ran a follow-up at $N{=}100$ per split to obtain bootstrap 95% confidence intervals. Each task was run with max_steps$=8$, temperature $0$, and a deterministic tool simulator. The aggregate results are in Table 6.

| Agent | Val | Sys | Prod | Sub | $\text{CGG}_{\text{prod}}$ | $\text{CGG}_{\text{sub}}$ |
|---|---|---|---|---|---|---|
| Template ReAct | 0.27 | 0.43 | 0.54 | 0.25 | $-0.27$ | 0.02 |
| Template Native FC | 0.27 | 0.15 | 0.20 | 0.25 | 0.07 | 0.02 |
| DeepSeek-ReAct (N=100) | 0.130 | 0.060 | 0.040 | 0.140 | 0.090 | $-0.010$ |
| DeepSeek-FC (N=100) | 0.240 | 0.080 | 0.070 | 0.210 | 0.170 | 0.030 |

Table 6: DeepSeek-V4-Flash validation at $N{=}100$ per split. 95% bootstrap CIs (1000 iter) are reported in the supplementary. $\text{CGG}_{\text{prod}}$ = $\text{TSR}_{\text{val}} - \text{TSR}_{\text{productivity}}$; $\text{CGG}_{\text{sub}}$ similarly.

**Findings.**

1. **The CGG is real, not a template artefact.** DeepSeek-V4-Flash, a 2026-era frontier model, exhibits the *same qualitative pattern*: TSR drops from 0.13 (val, ReAct) to 0.04 (productivity) and from 0.24 (val, FC) to 0.07 (productivity). The compositional gap is not closed by richer LLM priors.

2. **LLMs underperform template heuristics on val.** DeepSeek-ReAct scored 0.13 on val vs. 0.27 for ReAct templates; DeepSeek-FC scored 0.24 vs. 0.27 for Native FC templates. The natural-language prompts are *harder* for LLMs than keyword-matched templates. Despite the lower val baseline, both DeepSeek agents collapse on compositional splits.

3. **Economics.** DeepSeek-V4-Flash costs approximately $0.0008 per task (4,000 tokens blended), so a full 3,000-task AgentCompose sweep costs under $2.40.

**Bootstrap confidence intervals.** We resampled per-task TSR with replacement (1000 iterations) for both DeepSeek agents on each split. The 95% CI half-widths are $\leq 0.10$ on every split at $N{=}100$, indicating the point estimates in Table 6 are stable.

This validation supports our central claim: compositional generalization is a fundamental bottleneck that is *not* solved by scale alone. Future work should:

1. Run the same evaluation with GPT-4 / Claude / Gemini as agents to confirm cross-model generalization.

2. Investigate whether fine-tuning on AgentCompose training data improves real-LLM compositional generalization.

We view template-based evaluation as a *lower bound* on the gap—if heuristic agents fail this badly, LLM agents likely fail similarly or worse on truly novel compositions.